SQL Code

```sql
show databases;

create database RetailDB_3;

use RetailDB_3 ;

show tables;

CREATE TABLE Customers (
customer_id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100),
city VARCHAR(50),
signup_date DATE
);

insert into Customers(customer_id, name, email, city, signup_date)values
(1,"Rohit Kumar",  "rohit.kumar@gmail.com",   "Delhi",      '2023-01-12'),
(2,"Sneha Sharma", "sneha.sharma@yahoo.com",  "Mumbai",     '2023-02-05'),
(3,"Amit Patel",   "amit.patel@gmail.com",    "Ahmedabad",      '2023-02-18'),
(4,"Priya Reddy",  "priya.reddy@gmail.com",   "Hyderabad",      '2023-03-02'),
(5,"Karan Singh",  "karan.singh@outlook.com", "Chennai",    '2023-03-15'),
(6,"Neha Verma",   "neha.verma@gmail.com",    "Pune",       '2023-04-01'),
(7,"Arjun Mehta",  "arjun.mehta@gmail.com",   "Bengaluru",      '2023-04-20'),
(8,"Ritika Gupta", "ritika.gupta@yahoo.com",  "Kolkata",        '2023-05-12'),
(9,"Vikram Joshi", "vikram.joshi@gmail.com",  "Lucknow",        '2023-05-25'),
(10,"Ananya Das",  "ananya.das@gmail.com",        "Bhubaneswar",'2023-06-08'),
(11,"Suresh Iyer", "suresh.iyer@gmail.com",   "Chennai",        '2023-06-20'),
(12,"Megha Kapoor","megha.kapoor@yahoo.com",  "Jaipur",     '2023-07-03'),
(13,"Ravi Shankar","ravi.shankar@gmail.com",  "Delhi",      '2023-07-15'),
(14,"Tanya Mishra","tanya.mishra@gmail.com",  "Noida",      '2023-08-01'),
(15,"Aditya Jain", "aditya.jain@gmail.com",       "Indore",     '2023-08-14');

select * from Customers;

CREATE TABLE Suppliers (
supplier_id INT AUTO_INCREMENT PRIMARY KEY,
supplier_name VARCHAR(100),
contact_email VARCHAR(100),
city VARCHAR(50)
);

insert into Suppliers (supplier_id, supplier_name, contact_email, city) values
(1,     "Reliance Digital",     "support@reliance.com", "Mumbai"),
(2,     "Croma Electronics",    "info@croma.com",       "Delhi"),
(3,     "Flipkart Seller Hub",  "sellers@flipkart.com", "Bengaluru"),
(4,     "Amazon India Vendor",  "vendor@amazon.in",     "Hyderabad"),
(5,     "Tata Cliq",    "support@tatacliq.com", "Pune"),
(6,     "Vijay Sales",  "sales@vijaysales.com", "Chennai"),
(7,     "Lifestyle Stores",     "info@lifestylestores.com",     "Kolkata"),
```

```sql
(8,     "Metro Electronics",    "metro@electronics.com",        "Ahmedabad"),
(9,     "Big Bazaar",   "help@bigbazaar.com",   "Indore"),
(10,    "D-Mart Suppliers",     "supply@dmart.in",      "Nagpur"),
(11,    "Shoppers Stop",        "contact@shoppersstop.com",     "Jaipur"),
(12,    "Snapdeal Vendors"      ,"vendor@snapdeal.com", "Noida");

select * from Suppliers;

CREATE TABLE Shippers (
shipper_id INT AUTO_INCREMENT PRIMARY KEY,
shipper_name VARCHAR(100),
contact VARCHAR(100)
);

insert into Shippers (shipper_id, shipper_name, contact) values
(1,     "BlueDart Express",     "support@bluedart.com"),
(2,     "DTDC Courier"  ,"help@dtdc.com"),
(3,     "Delhivery Logistics",  "contact@delhivery.com"),
(4,     "Ekart Logistics"       ,"care@ekart.com"),
(5,     "Gati Express", "service@gati.com"),
(6,     "India Post Speed",     "info@indiapost.gov.in"),
(7,     "Shadowfax Delivery",   "support@shadowfax.in");

select * from Shippers;

   CREATE TABLE Products (
product_id INT AUTO_INCREMENT PRIMARY KEY,
product_name VARCHAR(100),
category VARCHAR(50),
price DECIMAL(10,2),
stock_qty INT,
supplier_id INT,
FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

insert into Products(product_id, product_name, category, price, stock_qty,
supplier_id) values
(1      ,"iPhone 15"                ,"Electronics"          ,"79999"        ,'50'   ,'1'),
(2      ,"Samsung Galaxy S24"       ,"Electronics"          ,"74999"        ,'40'   ,'2'),
(3      ,"Noise Smartwatch"         ,"Wearables"            ,"4999"         ,'100' ,'3'),
(4      ,"Boat Earbuds"             ,"Wearables"            ,"2499"         ,'200' ,'4'),
(5      ,"Kurta Set"                ,"Fashion"          ,"1999"   ,'150' ,'7'),
(6      ,"Running Shoes"            ,"Fashion"          ,"2999"   ,'120' ,'7'),
(7      ,"Prestige Mixer Grinder"   ,"Home Appliances" ,"6499"   ,'80'   ,'5'),
(8      ,"Sony Bravia TV"               ,"Electronics"      ,"59999"        ,'30'   ,'6'),
(9      ,"Lenovo Laptop"            ,"Electronics"          ,"55999"        ,'45'   ,'2'),
(10     ,"Philips Trimmer"          ,"Personal Care"        ,"1499"         ,'90'   ,'8');

  select * from Products;

CREATE TABLE Payment_Methods (
payment_id INT AUTO_INCREMENT PRIMARY KEY,
payment_type VARCHAR(50) UNIQUE
);
```

```sql
insert into Payment_Methods (payment_id ,payment_type)values
(1      ,"UPI"),
(2      ,"Credit Card"),
(3      ,"Debit Card"),
(4      ,"Net Banking"),
(5      ,"Cash"),
(6      ,"Wallet"),
(7      ,"EMI");
select * from Payment_Methods;

drop database retaildb_3;

CREATE TABLE Orders (
order_id INT AUTO_INCREMENT PRIMARY KEY,
customer_id INT,
order_date DATE,
payment_id INT,
shipper_id INT,
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
FOREIGN KEY (payment_id) REFERENCES Payment_Methods(payment_id),
FOREIGN KEY (shipper_id) REFERENCES Shippers(shipper_id)
);
insert into Orders(order_id,    customer_id,    order_date,     payment_id,
shipper_id)values
(1,       '1',  "2024-01-05",'1',       '3'),
(2,   '2',      "2024-01-06",'2',       '5'),
(3,       '3',  "2024-01-07",'4',       '2'),
(4,       '4',  "2024-01-08",'3',       '1'),
(5,       '5',  "2024-01-08",'5',       '7'),
(6,       '6',  "2024-01-09",'7',       '4'),
(7,       '7',  "2024-01-09",'6',       '6'),
(8,       '8',  "2024-01-10",'2',       '3'),
(9,       '9',  "2024-01-11",'1',   '5'),
(10,      '10', "2024-01-11",'3',       '2'),
(11,  '11',     "2024-01-12",'4',       '1'),
(12,  '12',     "2024-01-13",'5',       '6'),
(13,  '13',     "2024-01-14",'6',       '4'),
(14,  '14',     "2024-01-14",'7',       '7'),
(15,  '15',     "2024-01-15",'1',       '2'),
(16,  '3',      "2024-01-15",'2',       '5'),
(17,  '7',      "2024-01-16",'4',       '6'),
(18,  '10',     "2024-01-17",'5',       '3'),
(19,  '2',      "2024-01-17",'6',       '1'),
(20,  '6',      "2024-01-18",'3',       '7'),
(21,  '8',      "2024-01-19",'1',       '4'),
(22,  '1',      "2024-01-19",'7',       '2'),
(23,  '4',      "2024-01-20",'2',       '6'),
(24,  '11',     "2024-01-21",'5',       '5'),
(25,  '9',      "2024-01-21",'3',       '7'),
(26,  '12',     "2024-01-22",'4',       '1'),
(27,  '14',     "2024-01-23",'6',       '4'),
(28,  '13',     "2024-01-23",'2',       '3'),
(29,  '5',      "2024-01-24",'1',       '6'),
(30,  '15',     "2024-01-25",'7',       '7'),
```

```sql
(31,  '8',      "2024-01-25",'6',       '2'),
(32,  '7',      "2024-01-26",'3',       '5'),
(33,  '2',      "2024-01-27",'4',       '1'),
(34,  '9',      "2024-01-27",'2',       '4'),
(35,  '10',     "2024-01-28",'1',       '6'),
(36,  '11',     "2024-01-29",'5',       '7'),
(37,  '14',     "2024-01-30",'7',       '3'),
(38,  '6',      "2024-01-30",'3',       '2'),
(39,  '3',      "2024-01-31",'6',       '5'),
(40,  '12',     "2024-02-01",'2',       '1'),
(41,  '1',      "2024-02-01",'4',       '6'),
(42,  '13',     "2024-02-02",'5',       '7'),
(43,  '15',     "2024-02-02",'7',       '4'),
(44,  '4',      "2024-02-03",'1',       '2'),
(45,  '5',      "2024-02-04",'3',       '6'),
(46,  '2',      "2024-02-05",'6',   '7'),
(47,  '7',      "2024-02-05",'5',       '1'),
(48,  '9',      "2024-02-06",'2',       '3'),
(49,  '8',      "2024-02-07",'1',       '4'),
(50,  '10',"2024-02-07" ,'7',   '2'),
(51,  '11',     "2024-02-08",'6',       '5'),
(52,  '6',      "2024-02-09",'3',       '7'),
(53,  '14',"2024-02-09" ,'2',   '6'),
(54,  '12',"2024-02-10" ,'5',   '1'),
(55,  '13',"2024-02-10" ,'7',   '4'),
(56,  '1',"2024-02-11"  ,'4',   '3'),
(57,  '15',"2024-02-12" ,'6',   '2'),
(58,  '5',"2024-02-12"  ,'3',   '7'),
(59,  '4',"2024-02-13"  ,'2',   '5'),
(60,  '2',"2024-02-14"  ,'1',   '6'),
(61,  '3',"2024-02-14"  ,'7',   '1'),
(62,  '7',"2024-02-15"  ,'6',   '4'),
(63,  '8',"2024-02-15"  ,'5',   '2'),
(64,  '9',"2024-02-16"  ,'4',   '7'),
(65,  '10',"2024-02-16" ,'2',   '6'),
(66,  '11',"2024-02-17" ,'3',   '1'),
(67,  '12',"2024-02-18" ,'6',   '5'),
(68,  '13',"2024-02-18" ,'1',   '4'),
(69,  '14',"2024-02-19" ,'7',   '3'),
(70,  '15',"2024-02-20" ,'5',   '2'),
(71,  '1',"2024-02-21"  ,'2',   '7'),
(72,  '2',"2024-02-21"  ,'3',   '6'),
(73,  '3',"2024-02-22"  ,'4',   '1'),
(74,  '5',"2024-02-23"  ,'6',   '5'),
(75,  '7',"2024-02-23"  ,'7',   '4');

select *from Orders;

CREATE TABLE Order_Items (
order_item_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT,
product_id INT,
quantity INT,
price_each DECIMAL(10,2),
```

```sql
FOREIGN KEY (order_id) REFERENCES Orders(order_id),
FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

insert into Order_Items(order_item_id, order_id, product_id, quantity, price_each)values
(1,         1,  1,      2,      15000),
(2,         2,  3,      1,      12000),
(3,         3,  5,      3,      2500),
(4,         4,  2,      1,      22000),
(5,         5,  7,      2,      1800),
(6,         6,  4,      1,      8000),
(7,         7,  6,      4,      1500),
(8,         8,  8,      2,      3500),
(9,         9,  10,     1,      7000),
(10,    10,     9,      3,      1200),
(11,    11,     2,      1,      22000),
(12,    12,     1,      2,      15000),
(13,    13,     4,      3,      8000),
(14,    14,     7,      1,      1800),
(15,    15,     5,      2,      2500),
(16,    16,     3,      2,      12000),
(17,    17,     6,      3,      1500),
(18,    18,     9,      4,      1200),
(19,    19,     8,      1,      3500),
(20,    20,     10,     2,      7000),
(21,    21,     1,      1,      15000),
(22,    22,     2,      3,      22000),
(23,    23,     5,      2,      2500),
(24,    24,     7,      1,      1800),
(25,    25,     6,      2,      1500),
(26,    26,     4,      1,      8000),
(27,    27,     3,      1,      12000),
(28,    28,     8,      2,      3500),
(29,    29,     9,      1,      1200),
(30,    30,     10,     2,      7000),
(31,    31,     1,      3,      15000),
(32,    32,     2,      1,      22000),
(33,    33,     7,      2,      1800),
(34,    34,     4,      1,      8000),
(35,    35,     6,      3,      1500),
(36,    36,     5,      2,      2500),
(37,    37,     3,      1,      12000),
(38,    38,     9,      2,      1200),
(39,    39,     8,      1,      3500),
(40,    40,     2,      2,      22000),
(41,    41,     1,      2,      15000),
(42,    42,     4,      1,      8000),
(43,    43,     6,      3,      1500),
(44,    44,     7,      2,      1800),
(45,    45,     10,     2,      7000),
(46,    46,     3,      1,      12000),
(47,    47,     5,      2,      2500),
(48,    48,     9,      3,      1200),
```

```
(49,    49,     8,      2,      3500),
(50,    50,     1,      1,      15000),
(51,    51,     4,      2,      8000),
(52,    52,     2,      1,      22000),
(53,    53,     7,      3,      1800),
(54,    54,     6,      2,      1500),
(55,    55,     5,      1,      2500),
(56,    56,     10,     2,      7000),
(57,    57,     8,      1,      3500),
(58,    58,     9,      2,      1200),
(59,    59,     3,      1,      12000),
(60,    60,     2,      2,      22000),
(61,    61,     1,      2,      15000),
(62,    62,     4,      3,      8000),
(63,    63,     7,      2,      1800),
(64,    64,     6,      2,      1500),
(65,    65,     5,      1,      2500),
(66,    66,     9,      3,      1200),
(67,    67,     8,      1,      3500),
(68,    68,     10,     2,      7000),
(69,    69,     2,      2,      22000),
(70,    70,     1,      3,      15000),
(71,    71,     3,      1,      12000),
(72,    72,     4,      1,      8000),
(73,    73,     5,      2,      2500),
(74,    74,     7,      2,      1800),
(75,    75,     6,      3,      1500);

select *from Order_Items;

  ###a) Joins, Group By, Order By, Aggregations:

### Q1. Find the total revenue collected by each shipper.

show tables;
select *from orders;
select *from shippers;
select *from customers;
select *from suppliers;
select *from products;
select shipper_name, sum(price*stock_qty) Revenue from orders O join shippers Sh on O.shipper_id=Sh.shipper_id join customers C on O.customer_id=C.customer_id join suppliers S on C.city=S.city join Products P on P.supplier_id=S.supplier_id group by shipper_name;

### Q2. Show the top 5 highest-spending customers along with their total payments.

select  * from Order_Items;
select name,sum(price_each*quantity) total_payments from orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id group by name order by total_payments desc limit 5;


### Q3. Find product categories where the average selling price is greater than 8000.
```

```
 select  category , avg(price_each*quantity) selling_price from products P join order_items ot on p.product_id=ot.product_id group by category having selling_price>8000;


### Q4. Show the total number of orders placed per city, sorted by highest to lowest.

select  city, sum(quantity) no_of_qty from products P join order_items ot on p.product_id=ot.product_id join suppliers S on P.supplier_id=S.supplier_id group by city order by no_of_qty desc;

### Q5. Find suppliers who supply more than 1 product category.

select supplier_name, count(prodmuct_name) no_of_product from suppliers S join products P on s.supplier_id=p.supplier_id group by supplier_name having no_of_product>1;

### Q6. Show each order along with the number of items it contains.

select order_id, quantity from Order_Items;

### b)  Subqueries – Nested & Correlated:
### Q7. Find customers who have spent more than the average spending of all customers.

select customer_id, sum(quantity*price_each) total_spend from orders O join Order_Items OT on O.order_id=OT.order_id group by customer_id having total_spend> (select avg(quantity*price_each) from Order_Items);


### Q8. List all products whose price is higher than the average product price in their category.

select product_name, price_each from  products P join order_items ot on p.product_id=ot.product_id group by product_name , price_each having price_each> (select avg(price_each) from order_items);


### Q9. Show customers who placed at least one order with total_amount greater than 50,000.

select * from orders;
select * from order_items;
select * from customers;
select name, sum(quantity*price_each) total_amount from customers C join orders O on C.customer_id=O.customer_id join
order_items OT on O.order_id=OT.order_id group by name having total_amount>50000 ;


### Q10. Find customers who placed more orders than the average number of orders per customer.

select name, sum(quantity) no_of_order from customers C join orders O on C.customer_id=O.customer_id join
```

```
order_items OT on O.order_id=OT.order_id where quantity>(select avg(quantity) from order_items) group by name;


### Q11. Find the most expensive product(s) in the catalog.

select product_name,price_each from order_items OT join products P on OT.product_id=P.product_id where price_each = (select max(price_each) from order_items) group by product_name,price_each;


### c)  Window Functions:

### Q12. Rank customers based on their total spending.

select name,sum(price_each*quantity) total_Spending, RANK() OVER (ORDER BY sum(price_each*quantity)  DESC) AS Salary_Rank
from orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id
 group by name order by total_Spending desc;

### Q13. Find cumulative sales amount by order date.

select * from order_items;
select* from orders;
select  order_date,
sum(price_each*quantity) Amount,
sum(sum(price_each*quantity)) over (order by order_date) as Cumulative_sales
from orders o join order_items  ot on o.order_id=ot.order_id group by order_date order by order_date;


### Q14. Get each customer’s order count and show their percentage contribution.

 select * from order_items;
 select customer_id,
 count(quantity) as T_Order,
 round(count(quantity)*100/sum(count(quantity)) over(), 2) as percentage_con
 from orders O join order_items OT on O.order_id=OT.order_id group by customer_id order by percentage_con desc;


### Q15. Show the most recent order per customer.

select * from customers;
select * from orders;
select* from(
select name, order_date,
row_number() over(partition by name order by order_date desc) as Last_order
from orders O join customers C on O.customer_id=C.customer_id) dudu where Last_order=1;


```

```
### Q16. List each product with its sales quantity and rank products within each category by total sales.

select Product_name,
category,
sum(quantity) T_Quantity,
sum(quantity*price_each) T_price, rank() over(partition by category order by sum(quantity) desc) as Top_Sale_Pro
from products P join order_items OT on P.product_id=OT.product_id group by product_name, category;


  ### d)    CTE & Case:

### Q17. Categorize products as ‘High’, ‘Medium’, or ‘Low’ price using CASE.

select * from products;
select * from order_items;

select product_name , price_each,
CASE
        when price_each > 20000 then 'High'
    when price_each <= 20000 and price_each >= 10000 then 'Medium'
    else 'Low'
    end as price_category
    from order_items OT join products P on OT.product_id=P.product_id group by product_name , price_each order by price_each desc;

show tables;


### Q18. Use a CTE to find top 3 customers by spending.

select name,sum(price_each*quantity) total_payments from orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id
group by name order by total_payments desc limit 3;


### Q19. Use a CTE with CASE to classify customers by loyalty (based on number of orders).

select name, sum(quantity) no_of_order,
case
        when sum(quantity)>10 then '5star_customers'
    when sum(quantity)<=10 and sum(quantity)>=7 then '4star_customers'
    else  '3star_customers'
    end as 'loyalty'
                from customers C join orders O on C.customer_id=O.customer_id join order_items OT on O.order_id=OT.order_id group by name order by sum(quantity) desc;

### Q20. Find monthly revenue growth percentage compared to the previous month.

SELECT
    month,
```

```
    Total_Revenue,
    ROUND(
        (Total_Revenue - LAG(Total_Revenue) OVER (ORDER BY month))
        / LAG(Total_Revenue) OVER (ORDER BY month) * 100,
        2
    ) AS growth_percentage
FROM
(select DATE_FORMAT(order_date, '%Y-%m') AS month,sum(quantity*price_each) Total_Revenue from orders O join order_items OT on O.order_id=OT.order_id
group by month) T order by month;

### Q21. Find top 2 customers per city based on total spending.

select * from customers;
select name,city,sum(price_each*quantity) total_payments from orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id
group by name,city;
SELECT name,city, total_spending
FROM (
    SELECT
        city,
        name,
        SUM(price_each*quantity) AS total_spending,
        DENSE_RANK() OVER (PARTITION BY city ORDER BY SUM(price_each*quantity) DESC) AS rnk
    FROM orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id
group by name,city
) ranked
WHERE rnk <= 2
ORDER BY city, total_spending DESC;


  ### e)   Miscellaneous Advanced Joins & Aggregations

### Q22. Find top 3 cities with highest sales revenue including shipper name.

select *from(
select city, shipper_name, SUM(price_each*quantity) AS total_spending
from orders O join customers C on o.customer_id=C.customer_id join Order_Items OT on O.order_id=OT.order_id join shippers S on S.shipper_id=O.shipper_id
group by city,shipper_name order by city) tcs order by total_spending desc limit 3;


### Q23. List all orders with customer name, product name, supplier, and shipper.

select order_id, name, product_name, supplier_name, shipper_name from orders O join shippers Sh on O.shipper_id=Sh.shipper_id join
customers C on O.customer_id=C.customer_id join suppliers S on C.city=S.city join Products P on P.supplier_id=S.supplier_id;


### Q24. Show total sales per supplier along with average order value.

```

```
select supplier_name,count(supplier_name), SUM(price_each*quantity)  AS total_sales
from orders O join shippers Sh on O.shipper_id=Sh.shipper_id join
customers C on O.customer_id=C.customer_id join suppliers S on C.city=S.city join
Products P on P.supplier_id=S.supplier_id join order_items ot on
ot.order_id=O.order_id
group by supplier_name ;

### Q25. Show product categories that contributed more than 30% of total sales
revenue.
```